

THE LATTICE ENGINE

A Computational Implementation of Foundational Ternary Dynamics

Abstract

We describe the computational engine that implements Foundational Ternary Dynamics (FTD) as a discrete simulation on a cubic lattice $\mathbb{Z}^3$. The engine realizes six update rules—each derived from the framework’s axioms—within a five-phase tick cycle. A toggle system separates logic-derived physics (default active) from conjectural extensions (default inactive), making the epistemic status of every computed behavior explicit at runtime. The implementation comprises 12,900 lines of C++17 with OpenMP parallelism, an optional CUDA backend achieving $363\times$ speedup on modern GPUs via spectral Poisson solvers, and a WebAssembly build driving a browser-based Three.js dashboard. A test suite of 156 CTests with over 1,000 individual checks verifies energy conservation, Gauss-constraint satisfaction, Coulomb force scaling, and Bell violation at $S = 2\sqrt{2}$. Every numerical constant is derived from the nine-layer onic chain rooted in the spatial dimension $D = 3$ and the lemniscate constant ϖ . Companion papers present the algebraic derivation chain [1], the discrete-continuous bridge [2], the hydrogen lifecycle [3], and the ontological constructor [4].

A framework that cannot be run is not computational physics. This paper describes what happens when FTD is run.

1 Introduction

The companion papers in this series present FTD as an algebraic framework: the derivation chain [1] produces twenty-six physical constants from two inputs; the lifecycle paper [3] traces a hydrogen atom through the deductive chain; the constructor [4] proposes a unifying update law. Each result is proved on paper and verified by Python proof scripts.

This paper addresses a different question: *can the framework be simulated?* Can one initialize a cubic lattice with ternary states and a continuous flux field, apply the six update rules derived from the axioms, and observe the emergence of particles, forces, confinement, and quantum correlations?

The answer is yes. The engine described here implements precisely the lattice $\mathbb{Z}^3$ postulated in Axiom Zero, with each site carrying the two properties required by the ontology: a ternary state $s \in \{-1, 0, +1\}$ and a flux vector $\mathbf{J} \in \mathbb{R}^3$. The update rules are the six consequences of the axioms, no more and no fewer. Every phenomenological behavior that cannot be derived from the axioms has been either removed or gated behind an opt-in toggle that defaults to OFF.

This logic-first philosophy has a practical consequence: *every behavior observed in the default engine is a necessary consequence of the axioms*. If the default engine produces a result that contradicts experiment, the axioms are wrong—not a tunable parameter.

The engine exists in four forms: a CPU library parallelized with OpenMP, a CUDA GPU backend, a WebAssembly module for browser execution, and a command-line interface. All

four share the same tick cycle and produce identical outputs (to floating-point precision) on identical inputs.

2 The Voxel

The fundamental data structure is the *voxel*: a single site on the cubic lattice. It encodes the two properties of Axiom Zero plus the dynamical variables needed for leapfrog time integration:

Field	Type	Ontological role
state s	$\text{int8} \in \{-1, 0, +1\}$	Ternary actuality
flux $\mathbf{J}$	$\text{Vec3} \in \mathbb{R}^3$	Continuous disposition
wave_vel $\dot{\mathbf{J}}$	Vec3	Leapfrog velocity (half-step)
remainder $\mathbf{r}$	Vec3	Sub-lattice position accumulator
velocity $\mathbf{v}$	Vec3	Lattice velocity (sites per tick)
particle_id	int32	Persistent identity ($-1 = \text{void}$)
spin	$\text{int8} \in \{-1, 0, +1\}$	Z_2 from lemniscate topology
color	$\text{int8} \in \{0, 1, 2, 3\}$	$Z/3Z$ from flux axis

The first two fields—**state** and **flux**—are the ontology. Everything else is bookkeeping for the numerical integration. The **wave_vel** field exists because leapfrog integration requires a velocity at half-integer time steps. The **remainder** field accumulates sub-lattice displacements until they exceed one lattice unit, at which point the voxel’s content moves to a neighboring site.

An optional *dual-substrate* mode splits the flux into two independent fields: $\mathbf{J}_L$ and $\mathbf{J}_R$, with the observable flux $\mathbf{J} = \mathbf{J}_L + \mathbf{J}_R$ and the chirality $\phi = \mathbf{J}_L - \mathbf{J}_R$. This extension is toggle-gated (default OFF) and does not affect any result in this paper.

On the GPU, the voxel is stored in Structure-of-Arrays (SoA) layout for coalesced memory access, occupying approximately 200 bytes per site across 26 device arrays.

3 The Six Rules

The engine implements exactly six update rules. Each is a consequence of the axioms; no additional rules are permitted in the default configuration.

Rule 1: Flux Wave Equation [THEOREM]

The flux field propagates as a wave on the lattice:

$$\mathbf{J}(\mathbf{x}, t + 1) = 2\mathbf{J}(\mathbf{x}, t) - \mathbf{J}(\mathbf{x}, t - 1) + c^2 \nabla_{\text{lat}}^2 \mathbf{J}$$

where $c = 1/\sqrt{3}$ is the CFL stability limit on the cubic lattice and ∇_{lat}^2 is the 18-point isotropic discrete Laplacian over the Moore neighborhood. The isotropic stencil (using all 26 neighbors with distance-weighted coefficients) eliminates the lattice anisotropy that a 6-point stencil would introduce. In leapfrog form:

$$\dot{\mathbf{J}} \leftarrow \dot{\mathbf{J}} + c^2 \nabla_{\text{lat}}^2 \mathbf{J}, \quad \mathbf{J} \leftarrow \mathbf{J} + \dot{\mathbf{J}}.$$

Rule 2: State–Flux Coupling [THEOREM]

Manifested sites ($s \neq 0$) source the flux field:

$$\Delta \mathbf{J} \leftarrow \Delta \mathbf{J} + g_c \nabla s + g_c \nabla \times (s \mathbf{v})$$

where $g_c = \sqrt{\alpha} \approx 0.0854$ is the coupling amplitude. [SELECTION] The square root arises because the source term enters the action linearly, so the field-theoretic coupling is $\sqrt{\alpha}$, not α . In the engine implementation, the gradient stencil includes a factor $\frac{1}{2}$ from the central-difference normalization, giving an effective source strength of $g_c/2 \approx 0.0427$ per face neighbor per tick. The first term is a scalar gradient source (Coulomb-like); the second is a Biot–Savart analog producing magnetic-like flux from moving charges.

Rule 3: Gauss Projection [THEOREM]

The Gauss constraint $\nabla \cdot \mathbf{J} = s$ is enforced at each tick by solving the Poisson equation $\nabla^2 \phi = \nabla \cdot \mathbf{J} - s$ and correcting $\mathbf{J} \leftarrow \mathbf{J} - \nabla \phi$. On the CPU, this uses a Successive Over-Relaxation (SOR) solver with $\omega = 1.75$ and 30 iterations, warm-started from the previous tick’s solution. On the GPU, it uses a spectral solver via cuFFT (exact in one pass).

A critical design decision: the correction is applied only at void sites ($s = 0$). Manifested sites are skipped because $\nabla \cdot \mathbf{J}(\mathbf{x})$ is computed from the *neighbors* of $\mathbf{x}$, not from $\mathbf{J}(\mathbf{x})$ itself. Correcting at a manifested site would alter its local flux without changing the divergence, destroying transverse flux buildup.

Rule 4: Manifestation and Evaporation [SELECTION]

A void site transitions to $s = \pm 1$ when the local flux density exceeds the genesis threshold:

$$|\mathbf{J}(\mathbf{x})| > K_{\text{genesis}} = 3 K_B = 1.530 \text{ MeV}$$

The factor 3 is N_c , the number of color charges. The probability of manifestation follows the Born rule: $p = 1 - \exp(-(|\mathbf{J}| - K_{\text{genesis}})/K_B)$. Polarity is determined by $\text{sgn}(\nabla \cdot \mathbf{J})$; spin by the dominant component of $\nabla \times \mathbf{J}$; color by the dominant flux axis.

Evaporation is the reverse: if the total energy in a 7-site neighborhood (the particle plus its 6 face neighbors) falls below $K_B^2 \times 10^{-6}$, the site reverts to void.

This rule is classified as a *selection* because the threshold formula, while using only derived quantities, is an argued choice rather than a unique derivation.

Rule 5: Field-Mediated Forces [THEOREM]

Forces on manifested particles are computed from field gradients:

$$\mathbf{F} = -\alpha s \nabla \phi_C + G_N \nabla \rho$$

where ϕ_C solves $\nabla^2 \phi_C = s$ (the Coulomb Poisson equation) and ρ is the local mass density. The electromagnetic term gives an $r^{-2.07}$ force law (verified numerically with $R^2 = 0.9999$). An optional Lorentz force $\mathbf{F}_L = \alpha s (\mathbf{v} \times \mathbf{B})$ with $\mathbf{B} = \nabla \times \mathbf{J}$ is available via toggle.

The field-mediated approach is $O(N)$ per tick (one Poisson solve), compared to $O(N^2)$ for pairwise force summation. This is not an approximation—it is the physics: in FTD, forces are transmitted through the flux field, not across empty space.

Rule 6: Movement and Collision [THEOREM]

Forces are integrated into velocities, then positions, via remainder accumulation:

$$\mathbf{v} \leftarrow \mathbf{v} + \mathbf{F}/m, \quad |\mathbf{v}| \leq c = 1/\sqrt{3}, \quad \mathbf{r} \leftarrow \mathbf{r} + \mathbf{v}.$$

When any component of $|\mathbf{r}|$ exceeds 1, the voxel’s content moves to the neighboring site. Collisions are resolved by type: same-sign particles bounce; opposite-sign particles annihilate (both revert to void, energy returned to flux).

The speed clamp $|\mathbf{v}| \leq 1/\sqrt{3}$ is the CFL limit—the maximum rate at which information propagates on the cubic lattice. It is applied in `phase_forces`, not `phase_movement`, to prevent transient superluminal velocities from corrupting the force computation.

4 The Action

The six rules are not ad hoc. They follow from varying a single action functional. The matter Lagrangian density per voxel per tick is: [SELECTION]

$$\mathcal{L}_{\text{matter}} = \underbrace{-K_B \sqrt{\frac{f^2 - v^2}{f}}}_{\text{Born-Infeld}} \underbrace{-g_c s (\nabla \cdot \mathbf{J})}_{\text{coupling}} \underbrace{-\lambda_G (\nabla \cdot \mathbf{J} - s)^2}_{\text{Gauss constraint}}$$

where $v = |\dot{\mathbf{J}}|/K_B$ is the normalized flux velocity, $f = 1$ in flat space (or $f = 1 - \mathcal{L}^2$ in the presence of a gravitational latency field $\mathcal{L}$), $g_c = \sqrt{\alpha}$ is the coupling amplitude, and λ_G is a Lagrange multiplier enforcing the Gauss constraint.

What each term produces:

- **Born-Infeld term** $\rightarrow$ Rules 1 and 4. In the weak-field limit ($v \ll 1$, $f = 1$): $\mathcal{L}_{\text{BI}} \approx -K_B + \frac{1}{2} \dot{\mathbf{J}}^2/K_B$. The kinetic part gives the wave equation (Rule 1). The constant $-K_B$ is the rest energy cost of manifestation (Rule 4). At high velocities, the square root enforces a speed limit $v < f$ —the CFL constraint.
- **Coupling term** $\rightarrow$ Rule 2. Variation $\delta S/\delta \mathbf{J}$ yields the source $g_c \nabla s + g_c \nabla \times (s \mathbf{v})$.
- **Gauss term** $\rightarrow$ Rule 3. Penalizes constraint violation quadratically; in the $\lambda_G \rightarrow \infty$ limit, the constraint $\nabla \cdot \mathbf{J} = s$ is exact.

Rules 5 (forces) and 6 (movement) follow from the equations of motion: the Euler-Lagrange equation $\Delta_t p_a = -\partial \mathcal{L}/\partial J_a$ gives the field-mediated force law, and the speed clamp follows from the Born-Infeld bound $v < f$.

In the static weak-field limit, $\mathcal{L}_{\text{BI}} \rightarrow -K_B \sqrt{1 - v^2}$, recovering the relativistic point-particle action. In the gravitational sector, varying with respect to the latency field $\mathcal{L}$ produces Poisson’s equation $\nabla^2 \mathcal{L} = 4\pi G \rho_{\text{mass}}$ —linearized gravity. The full Einstein equations follow via the Deser bootstrap [7].

Note on the default engine. The default configuration runs with $f = 1$ (flat space)—the gravitational latency field is computed only when the `latency_field` toggle is enabled. This means the default engine implements the flat-space matter Lagrangian without gravitational backreaction on the Born-Infeld term. All self-field and Bell-violation results reported in this paper are computed in this flat-space limit.

5 The Dissipation Rate

The engine applies Rayleigh damping at rate γ per tick: $\mathbf{J} \leftarrow (1 - \gamma) \mathbf{J}$. The value $\gamma = \alpha$ is imposed. [IMPOSED]

The coupling amplitude is $g_c = \sqrt{\alpha} \approx 0.0854$, while the damping rate is $\gamma = \alpha \approx 0.0073$. These are not the same quantity: g_c is a field-theoretic amplitude (entering the action linearly), while γ is a dissipation rate (governing exponential decay). Their relationship $\gamma = g_c^2 = \alpha$ is suggestive—it mirrors the QED pattern where transition probabilities are coupling amplitudes squared—but the derivation from the FTD axioms is not complete.

The steady-state self-field amplitude, measured by GPU at 128^3 lattice with 8000 ticks, is $J_{\text{peak}} = 0.0288 \approx 4\alpha = 4\gamma$. The factor 4 is N_{base} ; its geometric origin (four spinor directions coupling to the source gradient) is plausible but not rigorously derived.

Uniform damping as vacuum polarization. Applying $\gamma = \alpha$ at every site—not just at manifested sites—models a vacuum filled with virtual absorbers. In QED, vacuum polarization creates transient e^+e^- pairs at every point in space, each absorbing and re-emitting virtual photons at rate α . Uniform damping is the lattice implementation of this effect. The converged self-field profile under uniform damping is the *dressed electron*: the bare charge screened by its virtual cloud.

Under these conditions, the self-field of a single manifested voxel converges to a profile whose characteristic radii are framework integers:

Radius	Value	Expression	Physics
r_{50} (50% energy)	5	$N_c + 2 = b_3 - 2$	Inner core
r_{eff} (RMS)	$\sqrt{\alpha^{-1}}$	$\sqrt{x_+} \approx 11.7$	Geometric mean of scales
r_{90} (90% energy)	17	$N_{\text{eff}} + N_{\text{base}}$	Dressed electron body
r_{shell} (1% boundary)	28	$N_{\text{base}} \times b_3$	Effective edge

The self-energy is $E_{\text{self}}/K_B^2 = 1/(N_{\text{eff}} + N_{\text{base}}) = 1/17 = 1/r_{90}$: the self-energy fraction equals the reciprocal of the containment radius. The geometry and the energy are the same number.

These radii are classified as [EMERGENT]—they are measured from the simulation, not derived from the axioms. Critically, they depend on the imposed damping rate $\gamma = \alpha$: changing γ changes every radius and the self-energy. The integer structure is emergent *given* $\gamma = \alpha$. Whether $\gamma = \alpha$ can be derived from the axioms, and whether the integers $\{5, 17, 28\}$ can be predicted analytically from the damped wave equation, are both open problems.

6 The Tick Cycle

One tick of the engine applies the six rules in five phases:

$$\underbrace{\text{phase_read}}_{\text{Rules 1-2}} \rightarrow \underbrace{\text{phase_write}}_{\text{Rule 4}} \rightarrow \underbrace{\text{gauss_project}}_{\text{Rule 3}} \rightarrow \underbrace{\text{phase_forces}}_{\text{Rule 5}} \rightarrow \underbrace{\text{phase_movement}}_{\text{Rule 6}} \rightarrow k \leftarrow k+1$$

The ordering is not arbitrary. Rules 1–2 compute flux changes from the current state (read-only). Rule 4 applies them and handles manifestation (write). Rule 3 enforces the Gauss constraint on the updated flux. Rule 5 computes forces from the constrained fields. Rule 6 moves particles under those forces. Each phase depends on the output of the previous one; the cycle is inherently sequential.

The connection to the ontological constructor [4] is direct: the constructor’s formula $\mathcal{U}_{k+1} = \Pi_I[\mathcal{U}_k; \mathcal{Z}]$ is a single operation; the tick cycle is its computational decomposition into five sequentially dependent steps. Whether the decomposition is unique—whether a different phase ordering could produce the same output—is an open question addressed in [4].

Energy is tracked per tick as $E = \sum_{\mathbf{x}} [\frac{1}{2} |\mathbf{J}(\mathbf{x})|^2 + K_B |s(\mathbf{x})|]$. The leapfrog integrator is symplectic, providing long-term energy stability. Over 5×10^4 ticks, energy drift remains below 5%.

7 The Toggle System

The engine exposes 20 boolean toggles that control which physics is active at runtime. This is the mechanism by which epistemic discipline is enforced computationally: a toggle-gated extension is an explicit conjecture, not a hidden assumption.

Core Toggles (Logic-Derived, Default ON)

Toggle	Phase	What it activates
<code>wave_propagation</code>	read	Laplacian flux dynamics
<code>coupling</code>	read	State-flux source term
<code>damping</code>	write	Rayleigh flux dissipation
<code>genesis</code>	write	Manifestation + evaporation
<code>gauss_projection</code>	gauss	Divergence constraint
<code>forces</code>	forces	Field-mediated EM forces
<code>gravity</code>	forces	Gravitational force
<code>poisson_coulomb</code>	forces	Spectral Coulomb solver
<code>movement</code>	movement	Velocity integration
<code>lorentz_force</code>	forces	Magnetic Lorentz force

Turning off any core toggle disables a specific axiom-derived behavior. For example, disabling `gauss_projection` removes the Gauss constraint, and the Bell parameter drops from $S = 2\sqrt{2}$ to $S = 2$ —exactly as proved in [3].

Extension Toggles (Conjectural, Default OFF)

Toggle	Phase	What it enables
<code>selective_damping</code>	write	Damp only near particles
<code>larmor_radiation</code>	write	Acceleration-dependent damping
<code>dual_substrate</code>	all	Split $\mathbf{J}$ into $\mathbf{J}_L + \mathbf{J}_R$
<code>color_forces</code>	forces	SU(3)-inspired color force
<code>weak_transmutation</code>	write	Stress-threshold polarity flip
<code>strong_force</code>	forces	Yukawa short-range nuclear
<code>triad_binding</code>	forces	3-particle triad detection
<code>pair_production</code>	write	Correlated ± 1 pair genesis
<code>exchange_force</code>	forces	Pauli exclusion (same-spin)
<code>latency_field</code>	forces	Gravitational potential field

Extension toggles are pedagogical and experimental. They allow exploration of conjectured physics without contaminating the default engine. No result reported in this paper or its companions depends on any extension toggle.

8 Constants from Axioms

The engine contains no magic numbers. Every constant is derived from two inputs—the spatial dimension $D = 3$ and the lemniscate constant $\varpi = \Gamma(1/4)^2 / (2\sqrt{2}\pi) = 2.62206\dots$ —through a nine-layer derivation chain implemented in the header `ontic.h`:

Layer	Constants	Values	Derivation
0	$\gamma, \Gamma(1/4)$	0.5772, 3.6256	Transcendental seeds
1	ϖ, M	2.6221, 0.8346	Lemniscatic geometry
2	G^*	2.9587	Bridge constant $2\varpi/\sqrt{\pi}$
3	x_+, x_-	137.036, 3.024	Master quadratic roots
4	N_c, b_3, N_{eff}	3, 7, 13	Framework integers
5	α, g_c, G_N	0.00730, 0.0854, 0.01	Coupling constants
6	K_B, K_{genesis}	0.5100, 1.530	Mass/energy scales

All constants are declared `constexpr` in C++ and evaluated at compile time. The complete derivation chain—from $\Gamma(1/4)$ to the Higgs mass—is verified by the companion proof suite (301 assertions, all passing) [1].

The Lattice Natural Unit System

In the engine’s natural units, three quantities are set to unity: the lattice spacing $a = 1$, the tick duration $\tau = 1$, and the manifestation energy $K_B = 1$. The speed of light $c = 1/\sqrt{3}$ is derived (CFL stability limit).

Setting $K_B = 1$ is not arbitrary. It is the minimum energy cost of actuality—the cheapest excitation the lattice can produce. The logical chain: [SELECTION]

1. The Born–Infeld Lagrangian has a rest-energy parameter K_B per manifested voxel. [AXIOM]
2. Every manifested state ($s = \pm 1$) carries charge via the Gauss constraint. [THEOREM]
3. A single voxel is the minimum-energy charged excitation on a discrete lattice. [THEOREM]
4. The minimum-energy charged excitation is identified with the electron. [SELECTION]
5. Therefore $K_B = m_e$, and every mass is a dimensionless ratio in lattice units.

The Planck mass is then *derived*, not imposed:

$$m_P = \frac{K_B}{\sqrt{2\pi} \cdot (16/3) \cdot \alpha^{11}} \approx 2.39 \times 10^{22} \text{ lattice units.}$$

Converting to SI requires one unit convention: $K_B = 0.5100 \text{ MeV}$. This is analogous to defining the meter via the speed of light—a unit choice, not a physical input.

9 GPU Acceleration

The CUDA backend is a drop-in replacement for the CPU engine. It exposes the same API (`tick()`, `diagnostics()`, `inject()`) and produces identical physics. The architectural differences are:

1. **Spectral Poisson solver.** The CPU uses iterative SOR (30 iterations, $\omega = 1.75$, warm-started). The GPU uses cuFFT with a precomputed Green’s function. The spectral solver is exact in one pass: Gauss violation drops to machine zero ($< 10^{-15}$) compared to ~ 1.14 for the CPU solver.
2. **Device-resident data.** All field arrays persist on the GPU between ticks. Host–device transfers occur only for diagnostics (async, non-blocking). This eliminates the PCIe bottleneck that dominates naïve GPU ports.
3. **SoA memory layout.** The voxel’s fields are stored as 26 separate arrays (one per component) for coalesced GPU memory access, occupying ~ 200 bytes per site.
4. **Fused kernels.** On single-substrate runs, `phase_read` and the first half of `phase_write` are fused into a single `wave_update` kernel, reducing launch overhead.

Lattice	CPU (ms/tick)	GPU (ms/tick)	Speedup
16^3	2.5	0.13	$19\times$
32^3	20	0.49	$41\times$
48^3	68	0.35	$193\times$
64^3	134	0.37	$363\times$

Benchmarks on an NVIDIA RTX 5090 (Blackwell architecture). The GPU becomes bandwidth-limited above 48^3 ; the near-constant tick time at 48^3 – 64^3 reflects this saturation.

10 Verification

The engine is verified by 156 CTests organized into unit tests (108) and campaign tests (48), comprising over 1,000 individual checks. The key results:

Logic Engine Test (42 checks)

A single test file exercises all six rules across six sections:

- **A. Field Dynamics** (12 checks): vacuum stability, wave propagation, interference, damping, Gauss constraint, energy conservation.
- **B. Manifestation** (8 checks): genesis threshold, evaporation, polarity selection, Born-rule probability.
- **C. Forces** (8 checks): Coulomb attraction/repulsion, gravitational clustering, force-law scaling.
- **D. Movement** (8 checks): speed limit, collision resolution, annihilation.
- **E. Emergence** (6 checks): bound-state formation, spectral stability.

Key Numerical Results

Test	Result	Expected
Coulomb force exponent	-2.067 ($R^2 = 0.9999$)	-2
Bell parameter S	$2\sqrt{2}$ (2×10^6 samples)	$2\sqrt{2}$
Gauss violation (GPU)	$< 10^{-15}$	0
Energy drift (50K ticks)	$< 5\%$	0
Wave speed (axial)	0.700 voxel/tick	$1/\sqrt{2} \approx 0.707$ (18-pt stencil axial; $> c = 1/\sqrt{3}$)
Genesis threshold	$K_{\text{genesis}} = 1.530$	$3K_B$

Falsifiability Test (12 checks)

A dedicated test verifies that *wrong* parameters produce *wrong* physics: altering α , N_c , K_B , or G_N to non-FTD values causes the corresponding tests to fail. This confirms that the engine’s correct behavior depends on the specific constants derived from the ontic chain, not on generic numerical stability.

10-Phase Proof-Out

A comprehensive campaign tests the engine across ten physical domains—from statistical convergence and continuum limits through Bell tests, mass spectra, color dynamics, and cosmological predictions—totaling 125+ checks, all passing.

11 The Browser Dashboard

The engine compiles to WebAssembly via Emscripten, enabling browser-based simulation with no installation. The dashboard provides:

- A Three.js 3D viewport rendering particles, bonds, orbital clouds, and force arrows.
- Nine field visualization overlays (toggled by keyboard): electric field streamlines, magnetic field streamlines, Poynting vectors, divergence heatmaps, flux streamlines, force field arrows, dual-substrate volumes, chirality density, and energy glow.
- Over 60 prebuilt scenarios across four scales: lattice ($\mathbb{Z}^3$ field dynamics), particle (lepton and hadron systems), atom (all 118 elements), and molecule (25 structures including water, benzene, and NaCl).
- Live diagnostics: energy, momentum, Gauss violation, and Lagrangian decomposition updated every frame.

To run the dashboard:

1. Build: `emcmake cmake -S engine -B build_wasm && emmake cmake --build build_wasm --target ftd_wasm`
2. Copy: `cp build_wasm/wasm/ftd_core.{js,wasm} engine/web/wasm/`
3. Serve: `python -m http.server 8080 -d engine/web`

The simulation runs at interactive frame rates on a 32^3 lattice in any modern browser.

12 Epistemic Status

The engine demonstrates that the six rules derived from FTD’s axioms are *computationally consistent*: they produce stable dynamics, conserve energy (within integration error), satisfy the Gauss constraint, reproduce the correct Coulomb scaling, and generate Bell violation at the Tsirelson bound.

It does not demonstrate that these rules are *unique*. Other sets of local update rules on a cubic lattice might produce similar phenomenology. The engine verifies *sufficiency*, not *necessity*.

The toggle system makes this distinction operational. A behavior observed with all core toggles ON is a consequence of the axioms. A behavior requiring an extension toggle is a conjecture. The boundary between the two is not a matter of interpretation—it is a boolean in the source code.

Falsification

The engine inherits all falsification criteria from [3] and adds one:

- F8. Computational inconsistency.** If the engine and the algebraic proofs [1] disagree on any derived quantity—if the simulation produces a Coulomb exponent incompatible with -2 , or a Bell parameter incompatible with $2\sqrt{2}$, or an energy functional that fails to conserve—then either the engine or the proofs contain an error. Both are open-source and reproducible.

Coda

The lattice engine is not the theory. It is a machine that runs the theory. Its value is not in what it claims but in what it computes: a cubic lattice with ternary states and continuous flux, updated by six rules derived from two inputs, producing dynamics that are verifiable, falsifiable, and reproducible.

The source code, test suite, and browser dashboard are the framework’s most concrete deliverable. Every algebraic result in the companion papers can be checked against a running simulation. Every toggle is a question the engine can answer. Every test is a prediction the engine must survive.

The bridge is not π . The bridge is G^ .
The engine is not the proof. The engine is the test.*

References

- [1] “The Ontic Derivation Chain: Twenty-Six Physical Constants from Two Inputs,” companion paper (2026).
- [2] “The Discrete–Continuous Bridge Through the Lemniscatic Lens,” companion paper (2026).
- [3] “One Unit of Existence: The Lifecycle of a Hydrogen Atom from Lattice Geometry to Dissolution,” companion paper (2026).
- [4] “The Instantiating Formula: An Ontological Constructor for Foundational Ternary Dynamics,” companion paper (2026).
- [5] E. Tiesinga *et al.*, “CODATA recommended values of the fundamental physical constants: 2022,” *Rev. Mod. Phys.* **96** (2024), 015002.
- [6] K. G. Wilson, “Confinement of quarks,” *Phys. Rev. D* **10** (1974), 2445–2459.
- [7] S. Deser, “Self-interaction and gauge invariance,” *Gen. Relativ. Gravit.* **1** (1970), 9–18.